

Agenda

- Past feedback and SBOM generation
- Literature Review and Identifying Gaps:
 - **On the Correctness of Metadata-based SBOM Generation: A Differential Analysis Approach**
 - **Enhancing Visibility of Components and Dependencies Across Diverse IT Environments with Open-Source Software-Bill-of-Materials Generation Tools**
 -
- Future directions

On the Correctness of Metadata-based SBOM Generation: A Differential Analysis Approach (1/2)

→ **Methodology of the paper entails:**

1. Selecting four popular SBOM generators: Trivy, Syft, Microsoft SBOM Tool, and GitHub Dependency Graph.
2. Collecting 7,876 open-source projects written in Python, Ruby, PHP, Java, Swift, C#, Rust, Golang and JavaScript.
3. Evaluating the correctness of the SBOMs by conducting a differential analysis on the outputs from these four tools.

→ **Key Findings:**

- Differential analysis showed significant variations in the number of dependencies reported and low Jaccard similarity between the dependency sets generated by different tool pairs for the same repository. This indicates tools often disagree on what dependencies are present. High rates (>10% on average) of **duplicate package** entries were also found.
- Tools generally fail to resolve transitive dependencies. They also have limited metadata support, parsing errors, and inconsistent naming. (**Parser Confusion Attack** demonstration)
- A small-scale ground truth test by dry running pip install for Python's requirements.txt showed very low recall (<10% for Trivy/Syft/GitHub DG), primarily due to lack of transitive dependency resolution and incomplete syntax support.

On the Correctness of Metadata-based SBOM Generation: A Differential Analysis Approach (2/2)

→ **Gaps:** Limitations of the study included relying on differential analysis (due to lack of ground truth dataset and benchmarks), focusing only on 4 (yet popular) SBOM tools, and evaluating accuracy only on Python's *requirements.txt*.

While the paper identified the *problem* of missed transitive dependencies, a gap exists in quantitatively measuring this failure (precision/recall) across multiple ecosystems using robust ground truth.

Furthermore, comparing the accuracy of these *metadata-parsing* tools against *build-integrated* tools (like CycloneDX plugins) specifically for transitive dependencies hasn't been systematically done.

Hence, the **core gap** is the lack of a focused, empirical measurement study quantifying the accuracy (precision/recall) of various SBOM generation approaches (metadata-parsing vs. build-integrated) specifically concerning transitive dependency detection across major ecosystems, using well-defined ground truth.